

CNN Training on CIFAR-10

Lab 2 Worksheet

Akanksha Kapil

M25CSA033

1 Objective

The objective of this experiment is to design and train a **Convolutional Neural Network (CNN)** on the **CIFAR-10 dataset**, analyze its computational complexity using **FLOPs**, and study the **training dynamics** through **gradient flow** and **weight update flow** visualizations using **Weights & Biases (WandB)**.

2 Dataset Description

The **CIFAR-10** dataset consists of **60,000 color images** of size $32 \times 32 \times 3$, belonging to **10 different classes**. The dataset is divided into:

- 50,000 training images
- 10,000 test images

The test set is used to evaluate the model's generalization performance during training.

3 Data Preprocessing and DataLoader

A **custom PyTorch DataLoader** was implemented for efficient data handling.

3.1 Training Data Transformations

- Random horizontal flip
- Random crop with padding
- Conversion to tensor
- Normalization with mean and standard deviation (0.5, 0.5, 0.5)

3.2 Test Data Transformations

- Conversion to tensor
- Normalization (no augmentation)

Shuffling was enabled for the training DataLoader to avoid bias during optimization, while the test DataLoader was not shuffled.

4 Model Architecture

A custom CNN architecture was designed and trained **from scratch** without using pre-trained weights. The model consists of:

- Four convolutional layers with Batch Normalization and ReLU activation
- Two max-pooling layers for spatial down-sampling
- Two fully connected layers for classification

This architecture balances **model capacity** and **computational efficiency**, making it suitable for CIFAR-10.

5 Computational Complexity (FLOPs)

The computational cost of the model was measured using the **THOP** library.

- **Approximate FLOPs:** 51.84 MFLOPs
- **Number of parameters:** 1.12 million

This indicates that the model is lightweight while still capable of achieving strong classification performance.

6 Training Setup

- **Optimizer:** Adam
- **Loss Function:** Cross-Entropy Loss
- **Batch Size:** 128
- **Number of Epochs:** 30
- **Hardware:** GPU (Google Colab)

The model was trained end-to-end with stable convergence observed throughout training.

7 Performance Results

After 30 epochs of training, the model achieved:

- **Training Accuracy:** 86.91
- **Test Accuracy:** 84.16
- **Final Training Loss:** ~ 0.40

The small gap between training and test accuracy indicates **good generalization** and minimal overfitting.

8 Visualization and Training Dynamics (WandB)

All training metrics and internal model behaviors were logged and visualized using **Weights & Biases (WandB)**.

8.1 Loss and Accuracy Curves

- Training loss decreased smoothly across epochs.
- Training and test accuracy increased steadily, indicating effective learning.
- Test loss followed a decreasing trend with minor fluctuations, which is expected in stochastic training.

8.2 Gradient Flow Analysis

- Gradient flow was visualized by tracking the mean absolute gradient magnitude for each layer.
- No vanishing or exploding gradients were observed.
- Deeper layers exhibited relatively higher gradient magnitudes, which is expected behavior.

8.3 Weight Update Flow Analysis

- Weight update flow was visualized by monitoring the evolution of mean absolute weight values.
- Weights changed smoothly over epochs without sudden spikes or stagnation.
- This confirms stable optimization and effective parameter updates.

All visualizations are available through the provided **WandB project link**.

9 Findings and Observations

- The CNN trained from scratch achieved strong performance on CIFAR-10.
- FLOPs analysis shows the model is computationally efficient.
- Gradient flow and weight update visualizations confirm stable backpropagation.
- Data augmentation and batch normalization helped improve generalization.
- No signs of severe overfitting were observed.

10 Conclusion

This experiment demonstrates that a well-designed CNN trained from scratch can achieve competitive performance on CIFAR-10. The use of WandB enabled detailed analysis of training behavior, providing insights into optimization stability, gradient propagation, and parameter learning.

11 Links

- **GitHub Repository (Branch):** https://github.com/AkankshaKapil/M25CSA033_Lab2_worksheet
- **WandB Project Link:** <https://wandb.ai/akankshakapil8-iit-jodhpur/CIFAR10-CNN-Lab2/runs/tti6642h>

Note:

- All visualizations are reported via WandB.
- No trained model files were pushed to GitHub.